# UNDERGRADUATE FINAL YEAR PROJECT REPORT

Department of Electronic Engineering
NED University of Engineering and Technology

# Development of Heterogeneous Air Quality Monitoring, Analysis and Prediction System

**Group Number:** **Batch:2022**

**Group Member Names:**

Ramish Hassan EL:22092

Rafay Ali Khan EL:22128

Jamal Raza Kazmi EL:22130

Muhammad Asfand Khan EL:22133

Approved By

…….

| | | |
|---|---|---|
| Dr. Sundus Ali | Ms. Saba Fakhar | Dr. Muhammad Imran Aslam |
| Associate Professor | Lecturer | Focal Person, IoT Lab |
| Project Supervisor | Co-Supervisor | Industrial Advisor |

# Author's Declaration

We declare that we are the sole authors of this project. It is the actual copy of the project that was accepted by our advisor(s) including any necessary revisions. We also grant NED University of Engineering and Technology permission to reproduce and distribute electronic or paper copies of this project.

| Signature and Date | Signature and Date | Signature and Date | Signature and Date |
|---|---|---|---|
| _____ | _____ | _____ | _____ |
| Ramish Hassan | Rafay Ali Khan | Jamal Raza Kazmi | Muhammad Asfand |
| EL-22092 | EL-22128 | EL-22130 | EL-22133 |
| ramishhassan2004 @gmail.com | anythingabout111 @gmail.com | jamalrazakazmi 123@gmail.com | asfandk3421 @gmail.com |

# Statement of Contributions

Mr. Asfand spearheaded the physical development of the system, taking full responsibility for the hardware architecture. He managed the entire electronics workflow, from designing the initial system schematics and selecting the appropriate components to routing the custom printed circuit board (PCB) layout. His work ensured that the microcontroller and sensor network were integrated onto a highly stable, production-ready physical platform.

Mr. Jamal focused on the structural packaging and the foundational documentation of the project. He arranged a custom 3D enclosure for the hardware, ensuring the physical device was durable, protected, and properly ventilated for accurate sensor readings. Alongside this, he took a leading role in drafting the initial technical report, setting up the framework for our project's documentation.

Mr. Ramish was responsible for the predictive intelligence and analytical capabilities of the system. He managed the complete data science pipeline, which involved cleaning the gathered environmental data, and training and optimizing the machine learning models. Beyond the technical modelling, he focused heavily on refining, structuring, and polishing the final technical report.

Mr. Rafay managed the end-to-end software stack and cloud connectivity for the ecosystem. He wrote the embedded firmware for the ESP32 microcontroller to handle sensor data acquisition, configured the Firebase cloud infrastructure for real-time streaming, and built the user-facing mobile application using Flutter. Additionally, he collaborated closely on the final synthesis and proofreading of the project documentation.

# Executive Summary

Rapid city growth has made air pollution a silent health crisis, but the way we track it is surprisingly outdated. Right now, most cities rely on massive and expensive government monitoring stations. While they are very accurate, they stay in one fixed place and are spread far apart, so they completely miss small pockets of bad air. They cannot detect sudden spiking in pollution at a busy intersection or the specific air quality inside a residential neighborhood. This leaves public health officials with a blind spot regarding what citizens breathe at street level.

You might expect modern smart devices to solve this, but the current landscape of cheap sensors is fragmented. Many existing prototypes suffer from connectivity issues. They either rely entirely on Wi-Fi, which traps them indoors, or they use expensive cellular data that limits how often they can report back. Worse, they often have a major design flaw where if the internet connection drops, the environmental data is simply lost forever. These missing gaps make accurate analysis nearly impossible. To bridge these gaps, we built a flexible and portable air quality monitoring system. Powered by an ESP32 microcontroller, our device is packed with advanced laser scattering sensors and specialized gas detectors. Rather than getting locked into a single network, it features a unique dual connectivity layer that automatically swaps between Wi-Fi when indoors and cellular GSM networks when outdoors. Most importantly, we built in a resilience layer.

This foolproof sync mechanism guarantees a completely unbroken dataset, giving us the high-quality historical data we need to train powerful predictive machine learning models. Ultimately, this project moves us past simply watching pollution happen. By combining flexible hardware with robust data protection and predictive analytics, it offers a scalable and affordable way to dynamically map the air we breathe. This gives Communities and officials a highly practical tool for proactive public health management.

# Acknowledgements

First, all praise is due to Almighty Allah for giving us the strength and focus to complete this project.

We are incredibly grateful to our supervisor, Dr. Sundus Ali, and our co-supervisor, Miss Saba Fakhar. Their doors were always open to us, and their guidance was crucial in helping us navigate the difficult phases of hardware design and software debugging. We truly appreciate the time they took to correct our mistakes and the constant motivation they provided when things didn’t go as planned.

We also want to thank our industrial advisor, Dr. Muhammad Imran Aslam. His practical advice helped us look at our project from a real-world perspective, ensuring our design wasn't just theoretical but useful for the industry.

# Table of Contents

# List of figures

# List of tables

# United Nations Sustainable Development Goals

The SDGs are the blueprint to achieve a better and more sustainable future for all. They address the global challenges we face, including poverty, inequality, climate change, environmental degradation, peace and justice. There is a total of 17 SDGs as mentioned below. Check the appropriate SDGs related to the project.

| | |
|---|---|
| ☐ No Poverty | ☐ Zero Hunger |
| ☐ Good Health and Well-Being | ☐ Quality Education |
| ☐ Gender Equality | ☐ Clean water and Sanitation |
| ☐ Affordable and Clean Energy | ☐ Decent Work and Economic growth |
| ☒ Industry, Innovations and Infrastructure | ☐ Reduced Inequalities |
| ☒ Sustainable Cities and Communities | ☐ Responsible Consumption and Production |
| ☒ Climate action | ☐ Life Below Water |
| ☐ Life on Land | ☐ Peace, Justice and Strong Institutions |
| ☐ Partnerships | |

# Similarity Index Report

Following students have compiled the final year report on the topic given below for partial fulfillment of the requirement for bachelor's degree in electronic engineering.

**Project Title: Development of Heterogeneous Air Quality Monitoring, Analysis and Prediction System**

| S.NO | Student Name | Seat Number |
|---|---|---|
| 1. | Ramish Hassan | EL-22092 |
| 2. | Rafay Ali Khan | EL-22128 |
| 3. | Jamal Raza Kazmi | EL-22130 |
| 4. | Muhammad Asfand Khan | EL-22133 |

This is to certify that Plagiarism test was conducted on complete report, and overall similarity index was found to be less than 20%, with maximum 5% from single source, as required.

Signature and Date

……....

Dr. Sundus Ali

# Sundus Ali

## Finalized_FYDP_Report

FYDP 1

### Document Details

Submission ID
trn:oid:::3618:140258145

Submission Date
May 24, 2026, 11:05 PM GMT+5

Download Date
May 24, 2026, 11:07 PM GMT+5

File Name
Finalized_FYDP_Report.pdf

File Size
5.9 MB

85 Pages

17,113 Words

103,232 Characters

# 11% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

## Filtered from the Report

- Bibliography
- Quoted Text

## Match Groups

**146** Not Cited or Quoted 11%
Matches with neither in-text citation nor quotation marks

**1** Missing Quotations 0%
Matches that are still very similar to source material

**0** Missing Citation 0%
Matches that have quotation marks, but no in-text citation

**0** Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

## Top Sources

7% Internet sources
4% Publications
10% Submitted works (Student Papers)

# Chapter 1 Introduction

## 1.1 Background

Fast industrial growth plus heavy traffic choked our cities, dropping air quality to dangerous levels. Smog, carbon monoxide, and volatile organic compounds now haunt every street corner. That is the reality. Static monitoring stations served us for decades, offering precise spectroscopic analysis to track these invisible killers. Truth be told, these bulky machines cost too much money to build anywhere but the wealthiest districts. Because cities only place them in sparse coordinates, huge gaps remain in our environmental data. Officials cannot address what they refuse to measure across shifting microclimates. Relying on fixed sensors creates dangerous blind spots for public health. Newer, mobile tech must bridge these gaps before urban centers suffer even deeper, irreversible damage to their fragile local atmosphere.

## 1.2 Motivation and Significance

### 1.2.1 The Urban Air Pollution Health Crisis

Public health concerns drove our team to launch this project. Fine particulate matter like PM2.5 and PM10, alongside carbon monoxide and carbon dioxide, triggers respiratory distress, heart failure, and permanent lung damage. Kids and seniors face the highest danger.

Cities like Karachi suffer immensely from heavy traffic and sprawling industrial zones. Rapid population growth makes matters worse, yet no one tracks the fallout properly. To be honest, the current infrastructure barely registers these extreme levels of pollution. That is the reality. Residents deserve clear data on what they breathe daily. Such ignorance serves no one in our modern, crowded urban world.

### 1.2.2 The Spatial Data Gap in Current Monitoring

Fixed stations often fail to capture nuanced pollution shifts at local levels. Air quality fluctuates wildly between adjacent streets, making distance-based readings unreliable.

Imagine a busy intersection flooding with exhaust while a distant sensor records clean air nearby. To be honest, that is the reality. Lacking granular, location-specific data keeps authorities from crafting responsive health warnings or environmental policies. Decisions based on broad averages miss the actual danger zones entirely. Street-level monitoring provides the missing clarity needed for change. It works better. Without precise facts, we remain blind to the risks hitting people in their own backyards.

### 1.2.3 Motivation for a Mobile and Predictive System

Urban environments demand systems capable of tracking pollution exactly where they accumulate. Mobility matters here. Mounting a monitoring unit onto vehicles or shifting positions across city streets offers a precise, comprehensive view of local air quality. That is the reality. Integrating machine learning shifts our approach from simple observation to forecasting future trends. Suddenly, we possess the ability to warn residents before conditions turn hazardous instead of merely reacting once the damage is finalized. Data collection becomes proactive guidance for everyone living in the area. Our project hinges upon this specific shift in logic. Everyone deserves cleaner air while navigating the daily grind.

## 1.3 Aims and Objectives

Building a heterogeneous air quality monitoring system remains our primary goal. We integrate environmental sensors, dual-mode communication, cloud storage, and machine learning into one mobile platform. The prototype handles complex data streams with ease. Simplifying real-time analysis matters. Keeping everything, portable makes field testing possible.

To achieve this primary aim, the project fulfills the following specific technical engineering objectives:

- To design and build portable sensing units that can work both indoors using Wi-Fi and outdoors using GSM/cellular connectivity, with GPS tagging on every measurement to record exactly where the reading was taken.

- To set up a cloud backend using Google Firebase Realtime Database that can receive, store, and display live sensor data through a mobile application.

- To develop a machine learning pipeline that trains on the collected sensor data and generates 24-hour ahead forecasts for air quality parameters, shown on an interactive dashboard with automatic alerts when thresholds are crossed.

- To test the full system through proper laboratory calibration and real-world testing in both indoor and outdoor environments, checking sensor accuracy, communication reliability, and overall system stability.

## 1.4 Methodology

The development of the portable air quality monitoring system followed an organized and systematic approach as depicted in Figure 1. The process initiated with an exhaustive literature survey and the identification of hardware components compatible with very essence of portability and real-time monitoring. All stages were planned, meticulously to fulfill the design requirements of the system, such as portability, energy efficiency, real-time monitoring, and reliability in an environment where it is necessary for a device to have the above-mentioned features.

The second phase included system integration and programming. The sensors set to be used in this project was chosen specifically to address the requirements of portability, battery efficiency, and cost-effectiveness. The third phase, the calibration of sensors was carried out successfully which helps in mitigation of issues involving sensor drift and environmental cross sensitivity. The fourth phase involved creation of PCB to replace the original breadboard prototype which not only reduced the wiring complexity but also enhanced the system robustness.

Once the hardware was finalized, a live dashboard mobile app was created to display sensor information in real-time. To enable remote monitoring, a cloud-based data storage was implemented in the project. The hardware was subsequently housed in a box to provide

stability of operation under outdoor field deployment. This was followed by an extensive series of outdoor tests assessing data consistency. GSM communication performance, and system overall robustness across different environmental conditions. Anomalies were debugged through iterative means.

This methodical strategy guaranteed that the final system was portable, durable, and capable of efficient, reliable, real-time monitoring of air quality in outdoor settings.

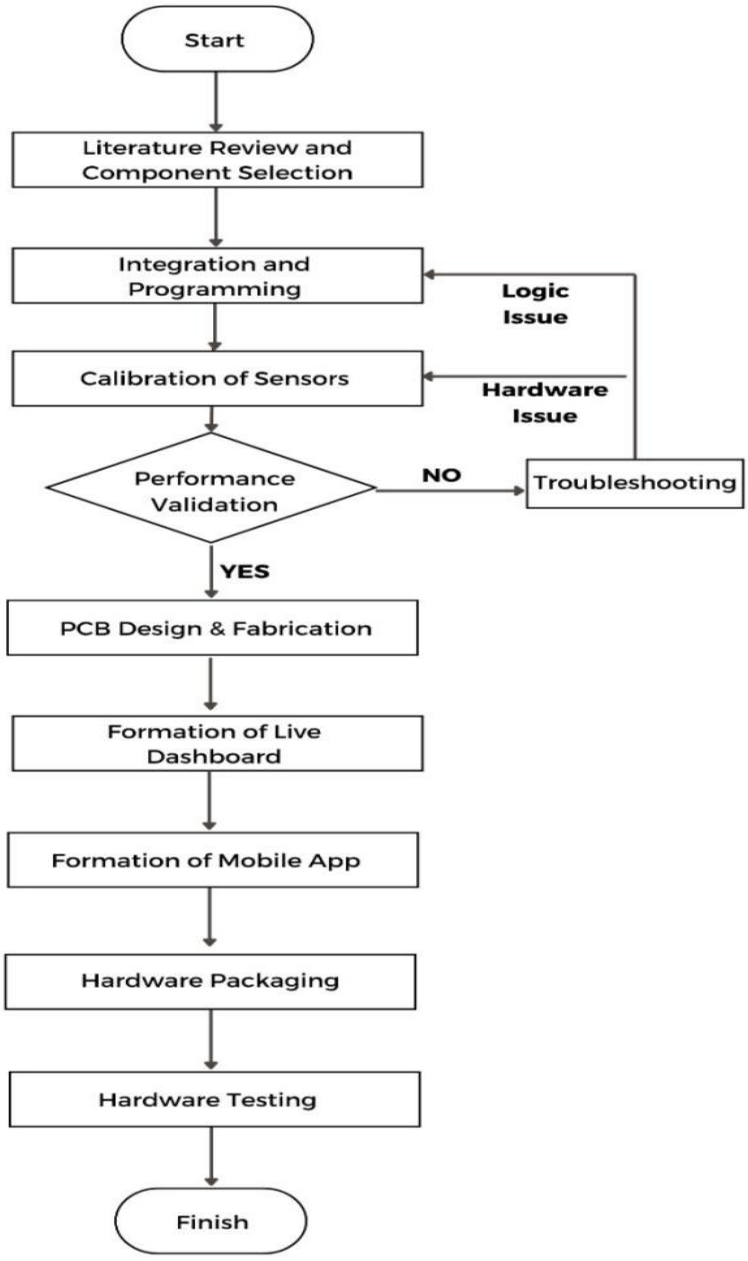


*Figure 1 Methodology chart*

### 1.4.1 Hardware Implementation

Integrating advanced sensors remains our primary goal here, specifically targeting those measuring CO, CO2, humidity, temperature, particulate matter and pressure. Connecting GPS and GSM hardware to the ESP32 microcontroller follows immediately. Picking the ESP32 made total sense for the integrated Wi-Fi capabilities. Calibrating these sensors ensures that our readings stay accurate under varying real-world conditions. We added a TFT display to show data in real-time for immediate feedback. Developing a dedicated battery management system allows for power optimization plus portability. Such hardware configurations allow the device to function effectively across different remote locations.

### 1.4.2 Predictive Analysis and Dashboard Visualization

Collected Sensor data is used to train a Random Forest regression model that generates 24-hour ahead predictions for all six monitored parameters. The model uses autoregressive lag features based on the last 15 minutes of readings for each parameter, which helps it understand the direction and rate of change in the environment. The results are shown in the "AQMS " Flutter mobile application. The dashboard displays live sensor readings, EPA-standard outdoor AQI scores, an indoor air quality index based on the Breeze Technologies model, a live GPS map, historical trend graphs, and realtime alert notifications.

### 1.4.3 Printed Circuit Board Designing

Upon entering the phase of PCB Designing, the team undertakes a methodical approach, beginning with the preparation of detailed schematic diagrams. Subsequently, a proper layout is created on the platform of EasyEDA, leading to the printing and implementation of the complete circuit on the PCB. This is iterative phase in the formation of this project. Thus, the implementation of this phase ensures the reliability and proper functionality of the system.

### 1.4.4 Final Testing and Deployment

The final testing phase involves performance testing of the integrated sensors in various environmental conditions. The functionality of the GPS module is verified for accurate

location tracking. Whereas in this phase, the device is deployed in diverse outdoor settings to assess the reliability and operational efficacy of the device.

## 1.5 Report Outline

Chapter 1 (Introduction) covers the background, motivation, objectives, methodology, and how the rest of the report is structured.

Chapter 2 (Literature review) reviews existing research on air quality monitoring, including traditional systems, IoT-based approaches, sensor technology, data management, machine learning, and the key gaps this project aims to fill.

Chapter 3 (System Design and Hardware Architecture) explains all the hardware decisions in detail, including which components were chosen and why, how the circuits are connected, how power is managed, and how the enclosures are designed for both indoor and outdoor use.

Chapter 4 (Software Development and Data Management) describes the complete software architecture, covering the ESP32 firmware, the Firebase cloud backend, and the Flutter mobile application, including how the AQI calculations and data resilience mechanisms work.

Chapter 5 (Machine Learning and Forecasting) explains the predictive pipeline from data collection and preprocessing through to model training, evaluation, and how the 24-hour rolling forecast is generated.

Chapter 6 (Conclusion and Future Work) summarizes what the project achieved, reviews each objective, discusses the limitations we encountered, and proposes directions for future improvement.

# Chapter 2 Literature Review

## 2.1 Introduction

This chapter gives us a detailed analysis of the current section of research on air quality monitoring technologies, with a central focus on the evolution fetched by the Internet of Things (IoT). The main goal of our project is to build a clear technological understanding through a critical review of existing literature, established methodologies and outstanding issues. This review gives us knowledge about different fields such as sensor technology, wireless communication, data systems, machine learning, and adoption factors that our project will need to address.

This review is presented clearly, step by step, to take the readers from basics to specifics of the tech areas and the larger ecosystem. It begins by drawing a clear comparison between the old monitoring methods and modern IoT-based systems. It then goes on to discuss the key technology building blocks: how devices talk to each other, the advances in sensors, how they are calibrated, how data is stored and managed, and how predictions are made. One section is dedicated to what stakeholders believe in and barriers within the system, essentially giving a complete view of deployment challenges. Finally, the chapter concludes with a summary that brings the findings together and clearly points out the research gap and why this proposed mixed monitoring and prediction system is needed.

## 2.2 Traditional Monitoring Systems and Their Limitations

For decades, the gold standard for tracking air quality has been massive, government-run monitoring stations packed with highly sensitive, extremely expensive chemical analyzers and particle balances. While these traditional setups are fantastic for generating the official reference data used to shape environmental policies, they have a glaring weakness: they cost an absolute fortune.

Building and maintaining just one of these stations runs into the hundreds of thousands of dollars, meaning even major cities can only afford a handful of them. This sparse, scattered setup creates huge blind spots in our data because pollution doesn't just sit evenly over a city; it changes dramatically from one block to the next, especially near busy intersections

or construction zones. Because these giant stations are usually parked in quiet, background locations to measure general trends, they completely miss the hyper-local pollution spikes happening right at street level. Plus, being bolted to the ground means they can't adapt or move to track sudden environmental events. Ultimately, relying entirely on these overly expensive, spread-out, and immovable stations just isn't enough anymore, which is exactly why there is such a desperate need to explore newer, more agile ways to monitor the air we breathe.

## 2.3 IoT-Enabled Air Quality Monitoring Systems

### 2.3.1 The IoT Framework

The proliferation of the IoT has transformed how large networks of inexpensive, connected sensors are developed. In environmental monitoring, the typical IoT infrastructure generally consists of four tiers: a sensing layer comprising low-cost sensors, a connectivity layer based on wireless protocols, a data layer in the cloud or a platform, and an application layer for visualization and analysis [11]. Most of the early works were deployed using Wi-Fi. One of the representative works is the Air-MIT system proposed by Messan et al. It was built with a NodeMCU (ESP8266) along with several MQ-series gas sensors, including MQ-135 and MQ-7 that track indoor air quality [12]. This system could detect a range of pollutants such as $CO_2$, CO, and $NH_3$ in real time and was also designed to trigger local actions like buzzer and exhaust fan upon the exceedance of limits. However, the design relies on Wi-Fi, which again restricts their applications to areas with stable internet facilities and makes the deployment mobile or outdoors over large areas problematic [12].

### 2.3.2 Connecting Over Large Distances

When developing our Universal Air Quality Monitoring and Prediction System, we quickly realized that capturing high-quality sensor data was only half the battle; the real challenge lay in ensuring that this data actually reached the cloud without any interruptions. To achieve this, we designed a highly adaptive networking architecture centred around our ESP32 microcontroller, prioritizing a robust dual-connectivity approach that leverages both Wi-Fi and GSM. We knew our system had to be truly universal, meaning it needed to perform just as flawlessly on a living room desk as it would mount on a street pole in a

busy neighborhood. When the device is deployed indoors or within the range of a stable local network, it naturally defaults to using Wi-Fi. This acts as our primary data highway, allowing for rapid, low-latency transmission of our PM2.5 readings and other environmental metrics directly to our backend while keeping power consumption relatively low. However, because air pollution is a highly dynamic problem that does not neatly confine itself to areas with broadband coverage, we integrated a dedicated GSM module into the hardware to act as our critical safety net. The moment our system detects that the Wi-Fi signal has dropped or is completely unavailable, which is entirely expected during outdoor deployments or mobile field testing, it automatically and seamlessly fails over to the cellular network.

This continuous, unbroken stream of data is absolutely vital for the predictive core of our project. Our forecasting models, specifically the Ridge Regression algorithmic engine we selected for its stability, rely heavily on a consistent feed of historical data to generate accurate 24-hour rolling forecasts; any dropouts in our network connectivity would directly compromise the integrity of these predictions. Beyond just transmitting the raw pollutant numbers, we also needed to give our data strict spatial context, which is exactly why we incorporated a precision GPS module into the hardware design. In modern environmental monitoring, it is simply not enough to know what the air quality index is at any given moment; we have to know exactly where that reading is taking place in order to accurately map out localized pollution hotspots. The GPS continuously tracks the physical coordinates of our unit, stamping every single packet of sensor data with exact latitude and longitude tags before it ever leaves the device.

All this carefully packaged data, the pollutant levels, the active network status, and the GPS coordinates, is then pushed directly to our Firebase cloud backend. We specifically selected Firebase because its real-time database capabilities serve as the perfect live bridge between our physical hardware out in the field and the end user. This lightning-fast synchronization is the engine that powers our custom mobile application, which we developed entirely from the ground up using the Flutter framework. Choosing Flutter allowed us to deliver a highly responsive, cross-platform experience that runs beautifully on any device, completely tethered to the live Firebase stream. It immediately updates the

user interface with the latest air quality indices, exact location mappings, and live environmental trends the very second the physical hardware records them. Furthermore, we wanted our tool to be practically useful for research and analysis, so we engineered a custom data export feature directly into the Flutter app. With just a tap, users can instantly compile and export all historical, geocoded sensor logs into clean, standardized CSV files formatted for easy parsing. This built-in CSV-making capability is incredibly powerful, as it allows anyone to seamlessly pull weeks of raw environmental data off the cloud and into their own spreadsheets or local machines for deep-dive statistical analysis or official academic reporting.

# 2.4 Sensor Selection, Tuning, and Data Integrity

## 2.4.1 Evolution of low-cost sensors

The outcome of Achievement in IoT air quality systems mostly depends on the sensors which are low cost and work best for these systems. The initial systems as they are low-cost metal-oxide sensors, such as the MQ series. Although, they are not expensive and are fast responding but the drawback is huge as their response to quite a lot of gases is not very different and their readings vary with temperature and humidity as a result their accuracy degrades over time, thus they require frequent re-calibration [15].

The latest systems use better sensor types to resolve these issues, thus resulting in obtaining more accurate data. In the case of dust particles, we use sensors such as PM2.5/PM10, researchers such as Abdullah et al. and Tanzila et al. have utilized the Plan tower PMS5003, which is a laser-based sensor that is significantly more accurate and stable compared with the older optical methods [13, 14]. For modern design systems related to gases like $CO_2$, Non-Dispersive Infrared (NDIR) sensors like the MH-Z19B are preferred, as they better target specific gases and remain reliable over time, compared to older electrochemical sensors [14].

## 2.4.2 The Imperative of Calibration

Even the very best low-cost sensors do not provide data as accurate as official, high-grade instruments. For this reason, calibration is not optional; it is a must in any system design.

Approaches to calibration vary; they can be as straightforward as applying a simple mathematical formula, for example, linear regression, to modify sensor readings using those from a nearby reference monitor, while others are more involved, using sophisticated machine learning (ML) techniques. For example, Ali et al. dealt with this by developing an ML calibration system. Computer models were trained using data from the reference monitors in order to make automatic corrections to the readings of inexpensive PM and gas sensors, and accuracy was thoroughly improved accordingly [16]. This represents a critical trend towards movement toward hybrid systems. Within these, smart software and data analysis greatly amplify what affordable hardware can achieve.

## 2.5 Data Management and Architectural Evolution

### 2.5.1 From Direct-to-Cloud to Tiered Architectures

The process of collecting and transmitting sensor data has become easier, yet the system is much more powerful. Initially, such systems used to be quite rudimentary. For instance, the Air-MIT configuration transmitted data directly from the sensor to a cloud site, such as Thing Speak, without any intermediate steps [12].

Later, a cellular IoT system added one middle step: the mobile sensors send their data as SMSs to a central receiver, or "gateway." This gateway gathers the data and sends it in one batch to cloud [13]. Newer designs, such as those used by Tanzila and colleagues with Lora WAN, are more structured and involve several layers. The data sequence is from sensors to a local gateway to a central network server (such as The Things Stack) to finally, a cloud application (such as Ubidots) [14].

In this modern setup, running the wireless network is separated from analyzing the data, and therefore the whole system scales more easily to more sensors and is more secure.

### 2.5.2 The Critical Gap: Data Resilience at the Edge

A major weakness in these cloud-based designs is that the data being sent to a cloud can be lost if the internet connection is poor. If the cellular signal is weak, the gateway may not function, or the network may be overloaded, causing the data to be transmitted to the cloud to be interrupted immediately. No system reviewed had a standard feature for reliably

storing data directly on the device, which is a major design flaw. That is a major design flaw. For some applications, such as machine learning, which require continuous data streams for training, and for accurate historical review, lost data can be very detrimental. A system cannot be considered strong if it cannot reliably save data locally, especially for mobile use or in areas with poor internet connectivity.

## 2.6 Predictive Analytics and Machine Learning Integration

### 2.6.1 From Descriptive to Predictive Monitoring

A major trend is towards the analytical capabilities of air quality systems. Predicting future levels of pollution can be crucial for enabling timely interventions, such as advance warnings to vulnerable people or managing discharges proactively [5]. While the prototypes reviewed did succeed in establishing the collection of data and viewing tools, they identified predictive machine learning as the key challenge and main direction for further work.

### 2.6.2 Models and Data Dependency

Long-term forecasting with temporal patterns can often be completed through time-series modeling. LSTM networks are particularly good at learning how temporal sequences relate to each other over long periods of time, which makes them well-suited for making predictions of future levels of PM2.5 or AQI based upon values observed within hours or days prior to the prediction date [17]. Hybrid CNN-LSTM models as well as CNN models can utilize the relationships between different sensor readings (spatial correlation) to make more accurate forecasts of both variables [18]. The success of all three types of models relies heavily upon the quality, quantity, and the continuous availability of historical data. Therefore, the relationship established earlier that clearly illustrates the significance of the previously established data resiliency gap; the need for an uninterrupted, high-quality source of historical data is fundamental to creating any successful predictive analytics model.

## 2.7 Stakeholder Perspectives and Ecosystem Challenges

The successful implementation of IoT-based AQMS extends beyond technical performance to encompass the broader ecosystem. Mariam et al. provided crucial insight through a regional study in Pakistan, analyzing the perceptions of stakeholders including academia, industry, government, and service providers regarding the use of IoT for climate action [19].

The study revealed a high awareness of IoT's potential benefits but pinpointed significant adoption barriers: high initial implementation costs, a shortage of local technical expertise for deployment and maintenance, serious concerns over data security and privacy, and the absence of comprehensive regulatory frameworks to ensure device interoperability, data standardization, and ethical data governance [19]. This research contextualizes the technical challenges, indicating that a viable system must be designed to be not only robust and accurate but also cost-effective, user-friendly, secure, and compliant with potential future regulations.

## 2.8 Summary and Identified Research Gap

### 2.8.1 Synthesis of Reviewed Literature

Looking closely at the existing literature, it is clear that Air Quality Monitoring Systems (AQMS) have undergone a massive evolution. They have transitioned from basic, stationary indoor sensors into highly sophisticated, wide-area IoT networks. Upgrades in communication protocols now allow these setups to cover much larger physical spaces, while better sensor hardware, coupled with machine learning calibration, means the data we collect is far more trustworthy. However, while real-time cloud dashboards are common today, most existing setups are highly specialized. They usually excel in one specific function but fall short in other critical areas.

### 2.8.2 Convergence of Unaddressed Needs

When reviewing these shortcomings, three major unaddressed needs become obvious. First, current designs suffer from monolithic connectivity. They rely entirely on a single communication protocol, like Wi-Fi, cellular, or LoRaWAN. Because of this restriction,

you cannot easily adapt the same device for both remote outdoor locations and indoor environments without risking network isolation.

Second, there is a widespread lack of data resilience. Very few current devices utilize local onboard storage. If the network goes down, the data in transit simply vanishes. It is virtually impossible to train a reliable machine learning model if your dataset is full of gaps caused by temporary connection drops. Finally, the industry lacks a fully connected predictive pipeline. Most systems fail to provide a seamless chain that starts with fault-tolerant data collection at the edge and ends with actionable, predictive visual analytics.

### 2.8.3 Research Gap

This project was launched specifically to bridge these gaps by developing a truly Universal Air Quality Monitoring and Prediction System. Building on the strengths of past research, this design introduces a comprehensive approach to solve these persistent issues.

Instead of being locked into a single network, the architecture integrates both Wi-Fi and GSM connectivity directly with the ESP32 microcontroller, creating a flexible communication layer that stays online regardless of the environment. To guarantee absolute data resilience, the hardware incorporates a dedicated local storage layer using an SD card. This ensures that even during network blackouts, data is continuously captured locally to maintain the complete, unbroken datasets required for advanced analytics.

Finally, the system establishes an intelligent, end-to-end pipeline. Raw sensor data moves seamlessly into edge storage, up to a cloud backend like Firebase, and feeds directly into an active predictive engine. By pivoting toward a stable linear approach and utilizing Ridge Regression as the final algorithmic engine, the system doesn't just log historical numbers; it actively forecasts future environmental conditions. Ultimately, this work delivers a much smarter, highly flexible tool that advances the fields of environmental sensing, IoT hardware design, and public health informatics.

# Chapter 3 Hardware Design and Implementation

## 3.1 Introduction

The Heterogeneous Air Quality Monitoring, Analysis, and Prediction System is the subject of this chapter for a detailed explanation of its overall hardware architecture and design. The overall design of the system follows the methodology described in Chapter 1, which will address the shortcomings of fixed monitoring systems [8] through the integration of a robust data resilient mobile sensing capability. Specifically, this chapter addresses the selection of the primary components of the system (the ESP32 microcontroller, the multi-sensor array, and the dual-mode communication modules), the interface between the various circuits, the methods used to manage the power consumption of the system and the mechanical design of the enclosures that are necessary to support the reliable operation of the system in both indoor and outdoor urban environments [9].

## 3.2 Finalizing the Components

Building a portable air quality monitoring system requires selecting parts that handle gas detection and environmental parameters with absolute precision. Every component must function reliably after toxic gases enter the local air stream. Picking the right hardware takes time. Getting the balance right is hard. Starting with the PM sensor, MQ7 and the MQ 135 Gas Sensor Module allows accurate CO2, CO and Particulate matters detection. ESP 32 Microcontroller units act as the core brain for processing incoming streams, while a DHT 22 Temperature and Humidity Sensor track surrounding weather shifts. Transmission needs depend on a SIM800L module, which pushes data across networks seamlessly. Visualizing these readings happens on a TFT Display, and a GPS Module locks onto your specific location during operation. Integrating these items creates a rugged, comprehensive device capable of tracking pollution patterns wherever you might travel. Each piece serves a distinct purpose, ensuring the final assembly meets every technical specification required for field success.

## 3.3 Integration of ESP32 with all the components

ESP32 functions as the main processor for our air quality monitoring system. Selecting this board provides dual-core power, plus full compatibility with various sensors and communication protocols. Connecting these modules allows real-time data processing and transmission of air quality readings. It captures accurate environmental data with absolute ease.

In this project MQ135 and MQ7 gas sensors as shown in Figure 2(a), is used to measure the $CO_2$ and CO. The ESP32 reads the sensor data which is then used to calibrate the Air Quality Index (AQI).


*Figure 2(a) MQ135 and MQ7*

Additionally, DHT22 sensor as shown in Figure 2(b), is used to measure temperature and humidity, with the VCC connected to 3.3V, Ground to GND and data pin to G4 on the ESP32. This system allows the monitoring of environmental conditions and the wireless transfer of data to a cloud server to analyze it in real time.

*Figure 2(b) DHT22 Temperature and Humidity Sensor*

PMS 5003 is a sensing device that can measure particulate matter in the atmosphere as shown in Figure 2(c). The sensor works as an optical particle counter which uses the lightscattering method to measure the concentration of airborne particulate matter in its environment. There is a fan that is built inside the sensor which draws the ambient air into its sensing chamber continuously. After the air is trapped inside the chamber, a laser beam of about 680nm laser is passed through the stream of air. Now if there are particles in the air, they will move and pass through that laser beam due to which the light gets scattered in different directions.

*Figure 2(c) PMS5003 Particulate matter sensor*

We use the ESP32 built-in Wi-Fi and Sim800L as shown in Figure 2(d) for robust communication and connectivity of our AQMS system to the main AQMS application and Firebase backend Server. Wi-Fi acts both indoors and outdoors but when Wi-Fi signals are lost the Sim800L replaces the Wi-Fi communication with cellular data so that sensor data don't get lost in the process and maintain the communication with ease.

*Figure 2(d) Sim800l GSM module*

To facilitate the creation of pollution heatmaps [9], every data point must be geotagged. The NEO-8M GPS module as shown in figure 2(e) communicates via UART to provide real- time Latitude, Longitude, and precise atomic time. This ensures that mobile readings can be mapped accurately to specific street locations.

*Figure 2(e) GPS Neo-M8n*

## 3.4 Integration of ESP32 with all the components

At the core of our system is the ESP32 microcontroller, which serves as the central point of connection for our many sensors and peripherals as shown in Figure 4. The TFT display will connect using SPI (with pins 15, 12, 2, 23, and 18) to create and continuously refresh the images used to present Current Readings on seven different pages that will be cycled through whenever the user presses button pin 32.

The DHT22 (on pin 4) will measure both temperature and humidity, while the BMP280 will measure atmospheric pressure through $I^2C$ (on pins 16 and 17). Figure 3 shows the hardware implementation of Air Quality Monitoring System.

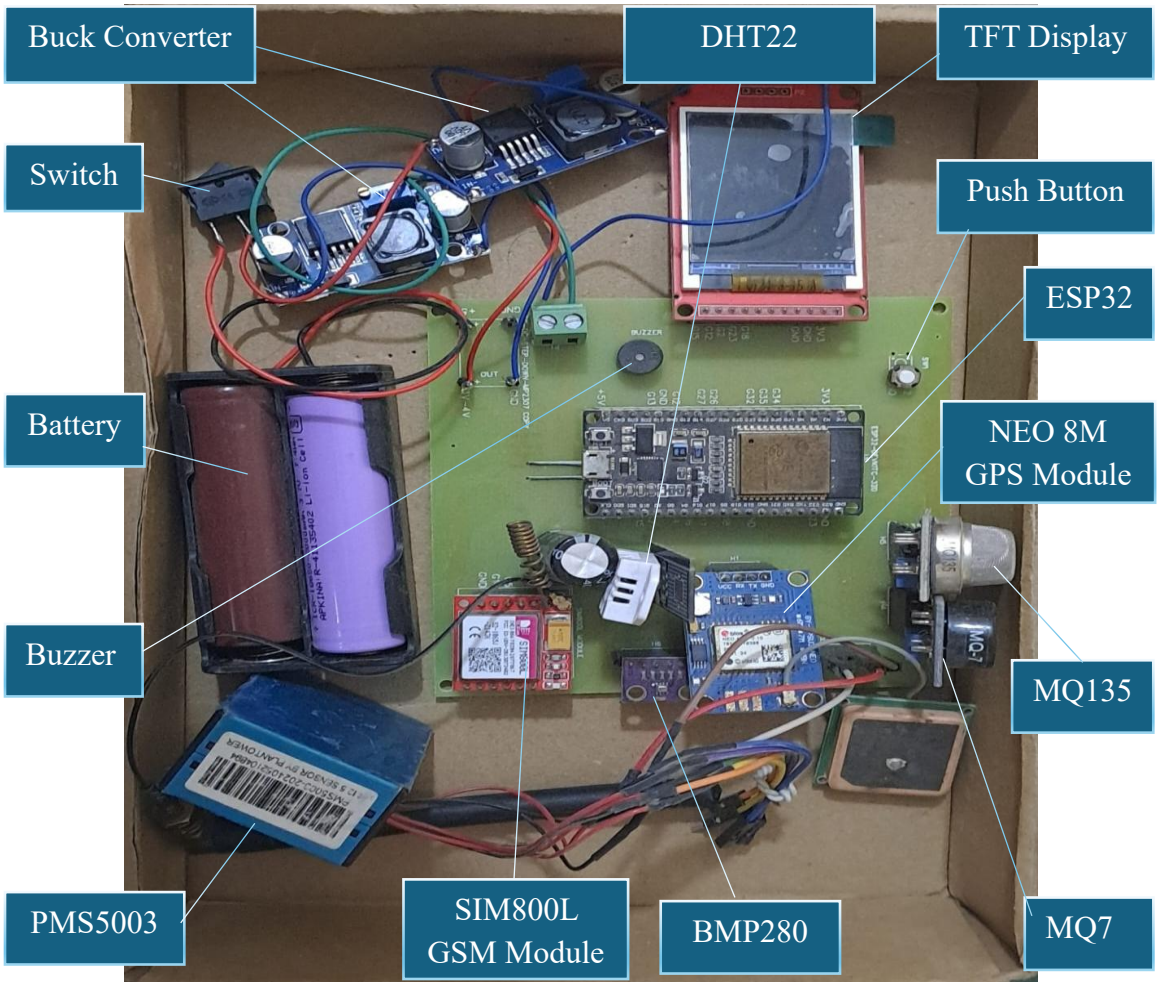


*Figure 3 Main circuit of Air Quality Monitoring System*

PMS5003 will report on the number of particulates in the air over UART2 (on pin 26). The MQ7 and MQ135 analog gas sensors (on pins 34 and 35) will require filtering of their measured values from analog to digital forms and will be converted to CO and $CO_2$ ppm, respectively.

The GPS module will connect over UART1 (on pins 27 and 25) and will return the current coordinates of the device. The buzzer (on pin 13) will sound an alarm anytime a measured value exceeds its configured threshold.

The time provided by the NTP server will be used to provide timestamps for all updates sent to Firebase so that Flutter can ignore timestamps that are too old. The entire system will read all connected sensors every 10 seconds, display the values locally.

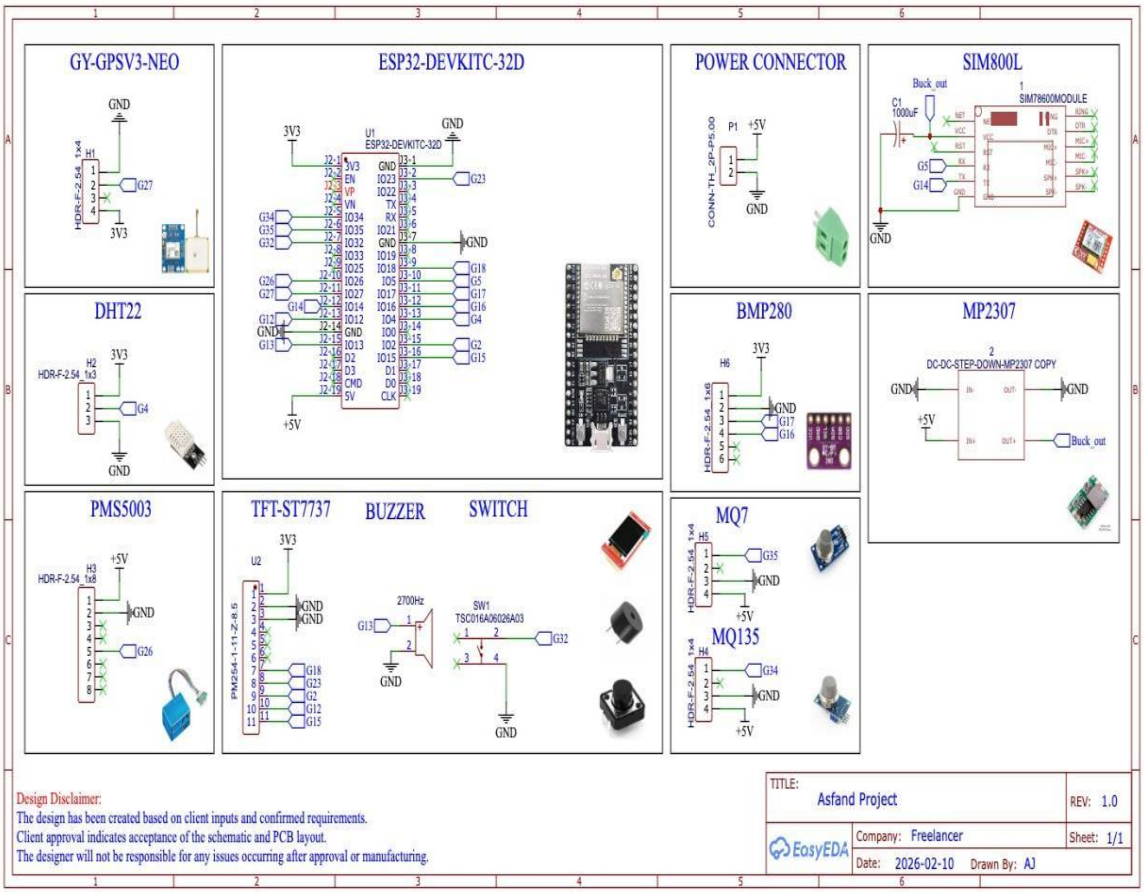


*Figure 4 ESP32 integration with sensors, modules and display*

## 3.5 PCB Design

As shown in Figure 5, the transition from a breadboard prototype to a customized Printed Circuit Board (PCB) was a critical step in ensuring the physical reliability and signal integrity of the proposed Air Quality Monitoring System (AQMS). The board was engineered to minimize electrical noise, manage power distribution efficiently, and provide a compact footprint suitable for mobile deployment.

The PCB layout incorporates several key design considerations:

- Centralized Processing Hub: The footprint for the ESP32 microcontroller is strategically placed to optimize trace lengths between the processing core and the various peripheral headers. This reduces parasitic capacitance and ensures high-

speed data integrity, particularly for the SPI communication lines routed to the ST7735 TFT display.

- Dedicated Sensor Interfaces: To accommodate the heterogeneous sensor array, specific terminal blocks and headers were mapped out. Isolated routing paths were designed for the analog signals of the MQ7 gas sensor to prevent crosstalk from high-frequency digital lines. Furthermore, dedicated UART channels were cleanly routed to interface with the PMS5003 particulate matter sensor and the u-blox NEO-M8N GPS module.
- Power Distribution and Grounding: Given the diverse power requirements of the system, especially the high current bursts drawn by the SIM800L GSM module during data transmission and the heating element of the MQ7 and MQ135 a robust power distribution network was implemented. Wide copper trace was utilized for the main power rails to handle higher current loads without overheating or voltage drops. Additionally, a comprehensive ground plane (copper pour) was applied across the board to provide a common reference point, drastically reducing electromagnetic interference (EMI) and stabilizing sensor readings.
- Modular Accessibility: The layout prioritizes modularity, with input/output headers positioned near the perimeter of the board. This design choice not only simplifies the final assembly and soldering process but also allows for easy troubleshooting, sensor calibration, or future hardware upgrades without requiring a complete board redesign.

Ultimately, this custom PCB design transforms the interconnected web of prototype wires into a rugged, deployable industrial-grade node, ensuring the AQMS can withstand vibrations and environmental stress during outdoor, on-the-go data collection.

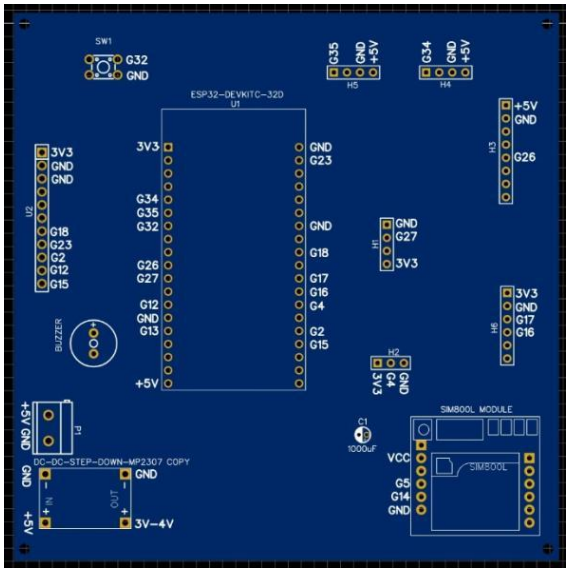


*Figure 5 PCB Design*

## 3.6 Power Management Unit

Stability of power is very important, especially for the GSM module that is subjected to large current spikes when transmitting data as noted in the module's design [25].

- **Voltage Regulation:** As this system utilizes a Lithium cell for portability, buck converters (step-down) were utilized to regulate the voltage supplied to the PMS5003, MQ135 and MQ7 sensors and ESP32 at 5V, as per standard power design practices [26].
- **Current Handling:** High-capacity capacitor was also installed near each of the GSM module power inputs to absorb the surge currents (up to 2A) needed for registration and SMS/data transmissions to prevent system brownouts.

## 3.7 Mechanical Design and Enclosure

The physical design of the product has been created to support the reliability of the hardware during field testing. The outer Body of AQMS is shown in figure 6, the outer body is made in blender software. It shows how the ESP32 works with all the components and communicate to application.

- **Ventilation**: The housing of the enclosure includes dedicated air intakes to create an environment where ambient air can flow easily around the sensors so as to prevent pockets of stale air that may affect the reading of the sensors [8].
- **Weatherproofing:** For the outdoor version of the unit, the housing will protect the electronics (ESP32, battery) from direct sunlight and light rain and still provide adequate ventilation to the sensors.
- **Portability:** To make the product portable, the size of the device has been minimized, allowing it to be handheld, mounted on a vehicle, etc., for use as a remote monitoring tool [9], [13].

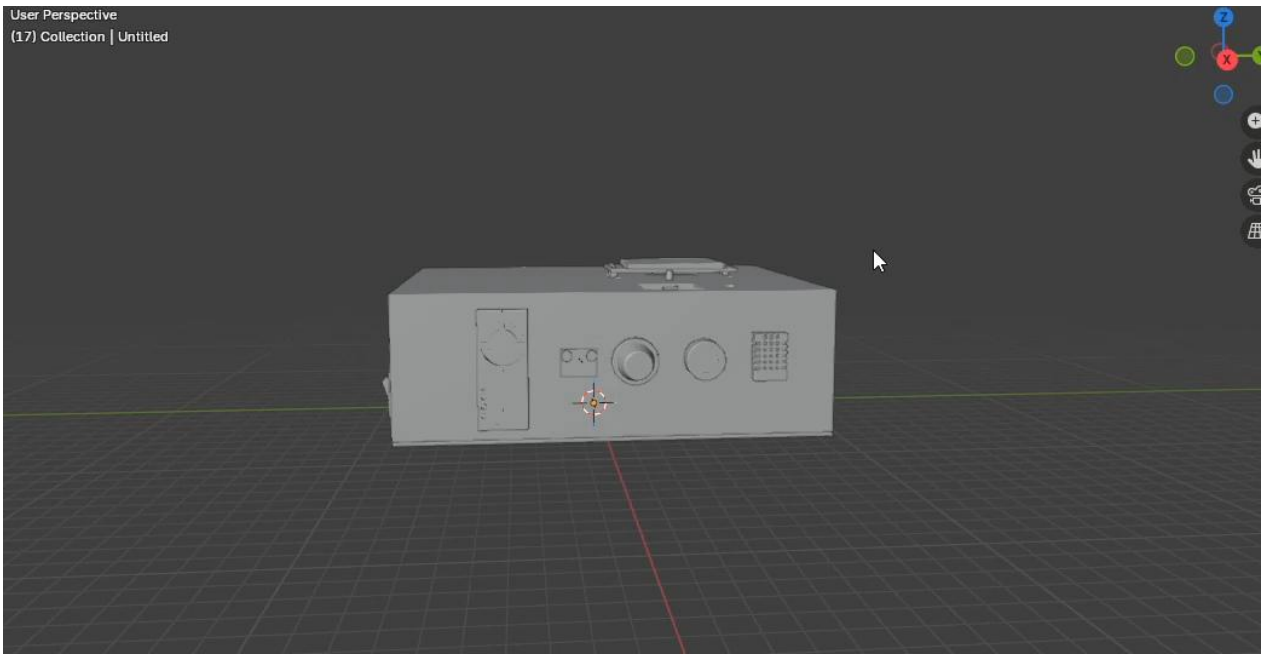


*Figure 6 Body of AQMS circuit*

## 3.8 Chapter Summary

The chapter above, this one describes how the hardware will support the objectives of the project. The hardware layer combines the processing capability of the ESP32 [20], the precision of the Laser Particle Sensors [21] and the mobility of GSM/GPS [13],[24],[25] to form a stable platform. With this design, we can create large amounts of quality, continuously collected data needed for the software analysis and predictive modeling [5],[18] as detailed in the next chapter.

# Chapter 4 Software Development and Data Management

## 4.1 Introduction

Well-designed hardware infrastructure, as discussed in Chapter 3, needs a corresponding software structure for sampling sensors, communicating, and providing valuable information to the users. In this chapter, the full software design architecture of the heterogeneous air quality monitoring system is detailed. The software architecture comprises three layers: the embedded firmware layer that executes on the ESP32 microcontroller, the cloud database that serves as the back-end service of Google Firebase Realtime Database, and the cross-platform mobile application designed using Flutter development framework. All the layers were implemented to resolve some of the gaps identified in the literature review, particularly on monolithic connectivity, data loss in case of network disruptions, and lack of an end-to-end predictive pipeline [11]-[14]. The description starts from the firmware layer and goes upward until reaching the user interface. The full implementation code is documented in the appendix section while the discussion in the following sections concentrates on the core algorithms and data structures.

## 4.2 Embedded Firmware Design (ESP32)

ESP32 firmware, which has been developed using C++ code inside the Arduino IDE, takes care of all the hardware peripherals, and it makes sure that no environmental measurement is ever lost. The main loop in the software consists of four timed operations: sensors readings, display update, sending to cloud, and alarms.

### 4.2.1 Sensor Data Acquisition and Software Filtering

The readAllSensors() function retrieves information from all the sensors. SensorReading structures are filled with the most recent values of temperature/humidity (from DHT22) and pressure (fromBMP280) through the use of the respective standard libraries; however, a PMS5003 particle sensor must use a hardware industrial-standard UART to transmit data. GPS positioning will be achieved using a second UART from a NEO 6M receiver and

corresponding data will be processed by TinyGPS++. The two-metal oxide-based gas sensors: MQ7 and MQ135 for CO and broad-based CO2 respectively, generate jittery analog voltages, which are conditionally filtered through a moving average filter of 5 values. The implementation for the filter.

The reference baseline values (baselineCO_Raw for the MQ-7 and baselineCO2_Raw for the MQ-135) are established during a one-time calibration routine performed at system startup. The ESP32 takes 20 successive analog readings from each gas sensor over 2 seconds while the sensors are exposed to clean, fresh air (e.g., outdoors or in a well-ventilated space). These 20 readings are averaged, and the resulting mean values become the clean-air baselines.

During normal operation, the current raw ADC value (RAW) for each gas sensor is the output of the moving average filter (5 samples) applied to the live analogRead() signal. This filtered RAW value is then substituted into the linear-ratio conversion equations as equations 1 and 2.

$$CO_{ppm} = 10.0 \times \left(\frac{rawbaselineCO}{RAW} - 1\right) \quad (1)$$

$$CO_{2ppm} = 400 + 100.0 \times \left(\frac{rawbaselineCO2}{RAW} - 1\right. \quad (2)$$

Equations 1 and 2 shows trade absolute accuracy for simplicity, yet they yield trends that correctly reflect pollutant changes, which is sufficient for threshold-based alerts and for training predictive models [16].

### 4.2.2 Real-Time TFT Display and Multi-Screen User Interface

The device consists of a 1.8-inch ST7735 TFT screen that can show you your sensor data without having to use your phone. The user has to push a button that is connected to pin #32 in order to cycle through the seven screens; a long press will calibrate the system. The screen will be updated by the SCREEN_INTERVAL timer (every 2 seconds). A Heartbeat indicator blinking Green LED located in the upper right corner and Red “ALARM” Label will be displayed when thresholds are exceeded. Local display was critical in performing field tests by providing instant verification of the sensor readings and communication.

### 4.2.3 Dual-Mode Connectivity (Wi-Fi / GSM) and Network Management

To escape the single network limitation observed in earlier systems [12, 13], the firmware uses the WiFiMulti library to connect to any of two pre-configured access points. If neither is found within a 15 second timeout, the device starts in offline mode and continues to log data locally. The loop monitors Wi Fi status and the firebase Ready flag, transitioning between online and offline states as shown in Figure 7. The use of transparent switching guarantees that the system will be able to function not only in Wi-Fi conditions but also in GSM environments in the future.

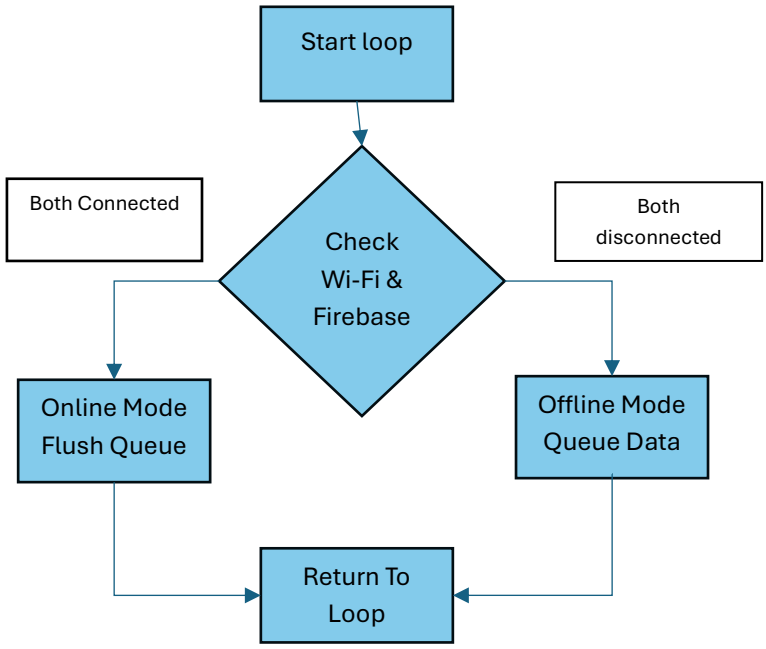


*Figure 7 Wi-Fi and Firebase connectivity management flow*

### 4.2.4 Firebase Realtime Database Integration and In-Memory Offline Queue

One of the key deficiencies of the AQMS systems developed by academia is their inability to store data locally when the Internet connection drops [13, 14]. In contrast, the proposed system uses an in-memory circular buffer for 50 instances of SensorReading. Should the Internet be disconnected, sendToFirebase() invokes addToQueue(), which stores the

instance in the memory. When Internet becomes available again, flushQueue() flushes the buffer into Firebase. The main code for the queue management is provided below.

Written to both /sensors/current (last reading) and/sensors/history/<timestamp> to archival storage. Combined with Firebase's own offline persistence layer, this queue provides a two-layer resilience strategy that provides near guaranteed data persistence as long as the device has power.

### 4.2.5 Threshold-Based Buzzer Alarms and Discord Webhook Alerts

The system checks sensors every second against WHO and ASHRAE standards. If the reading exceeds med or max safe values, a buzzer will sound for a maximum of 10 seconds. Simultaneously, an alert will be sent as a Markdown-formatted message via Discord webhook. The sound occurs only when the situation indicates danger.

At the end of every 5 minutes, a complete update on all sensors will be sent via HTTPS POST request to a pre-existing webhook. In situations where alarms trigger, a message will be sent immediately, providing an opportunity for team members to maintain their connection to their remote colleagues via live updates being sent into Discord. No additional servers are required by remote workers. The level of awareness remains high when team members are spread out across the globe. Table 1 shows the complete alert thresholds and notification behavior.

### 4.2.6 Local HTTP Server for Debugging

Once the Wi-Fi is connected, the system runs its code via small web servers on port 80 - that makes update easy. From there a request sent from HTTP tool (with CURL) to the /data address returns a JSON packet of live sensor readings attached with alert flags. Due to that setup sensor output check became very quickly, we can check sensor output from any browser just clicking the link - no need to login into Firebase.

*Table 1 Complete alert thresholds and notification behavior*

| Parameters | Threshold (exceedance triggers alert) | Local action (buzzer) | Remote notification |
|---|---|---|---|
| Temperature | ≥ 40.0 °C | Sound for max 10 seconds | Immediate Discord message |
| Humidity | ≥ 85.0 % | Sound for max 10 seconds | Immediate Discord message |
| Carbon Monoxide (CO) | ≥ 9 ppm | Sound for max 10 seconds | Immediate Discord message |
| Carbon Dioxide ($CO_2$) | ≥ 800 ppm | Sound for max 10 seconds | Immediate Discord message |
| PM1.0 | ≥ 50 µg/m³ | Sound for max 10 seconds | Immediate Discord message |
| PM2.5 | ≥ 150 µg/m³ | Sound for max 10 seconds | Immediate Discord message |
| PM10 | ≥ 250 µg/m³ | Sound for max 10 seconds | Immediate Discord message |

## 4.3 Cloud Backend & Firebase Realtime Database

### 4.3.1 Database Schema and Real-Time Synchronization

The database was selected over a custom Flask server described in the methodology section of this paper because of the native real time synchronization and the client libraries for both ESP32 and Flutter.

The database contains three nodes:

- **/sensors/current:** one JSON object, the latest reading, overwritten every 10 seconds.

- **/sensors/history:** a set of timestamped readings, with the device millis() value as a key to facilitate queries in chronological order, or export to csv.
- **/device_tokens:** a set of FCM registration tokens for future push notifications.

The flat structure allows the mobile app to listen to real time updates on /sensors/current and the history node to act as a training corpus for machine learning, as well as a safety net for calibration analysis.

### 4.3.2 Security Rules and Legacy Token Authentication

The prototype uses a legacy token built into the firmware to provide access to the database. The legacy token is part of the Firebase Config object and prevents clients from writing or reading from the database unless they are an authorized client. In a production environment, this would be replaced by Firebase authentication; however, the legacy token is secure enough for this project's scope.

### 4.3.3 Real-Time Data Flow from ESP32 to Flutter App

The end-to-end synchronization pathway is illustrated in Figure 8. The ESP32 reads all sensors, checks network availability, and either pushes the reading directly to the two database nodes (/sensors/current and /sensors/history) or if offline, stores the reading in a circular buffer for later transmission. Once the data lands in the Firebase Realtime Database, any connected Flutter client (foreground app or background service) receives the update instantly. The app’s currentRef.onValue listener updates the live dashboard, while the background service monitors the lastSeen field to detect device disconnections and triggers local notifications when sensor readings exceed critical thresholds.

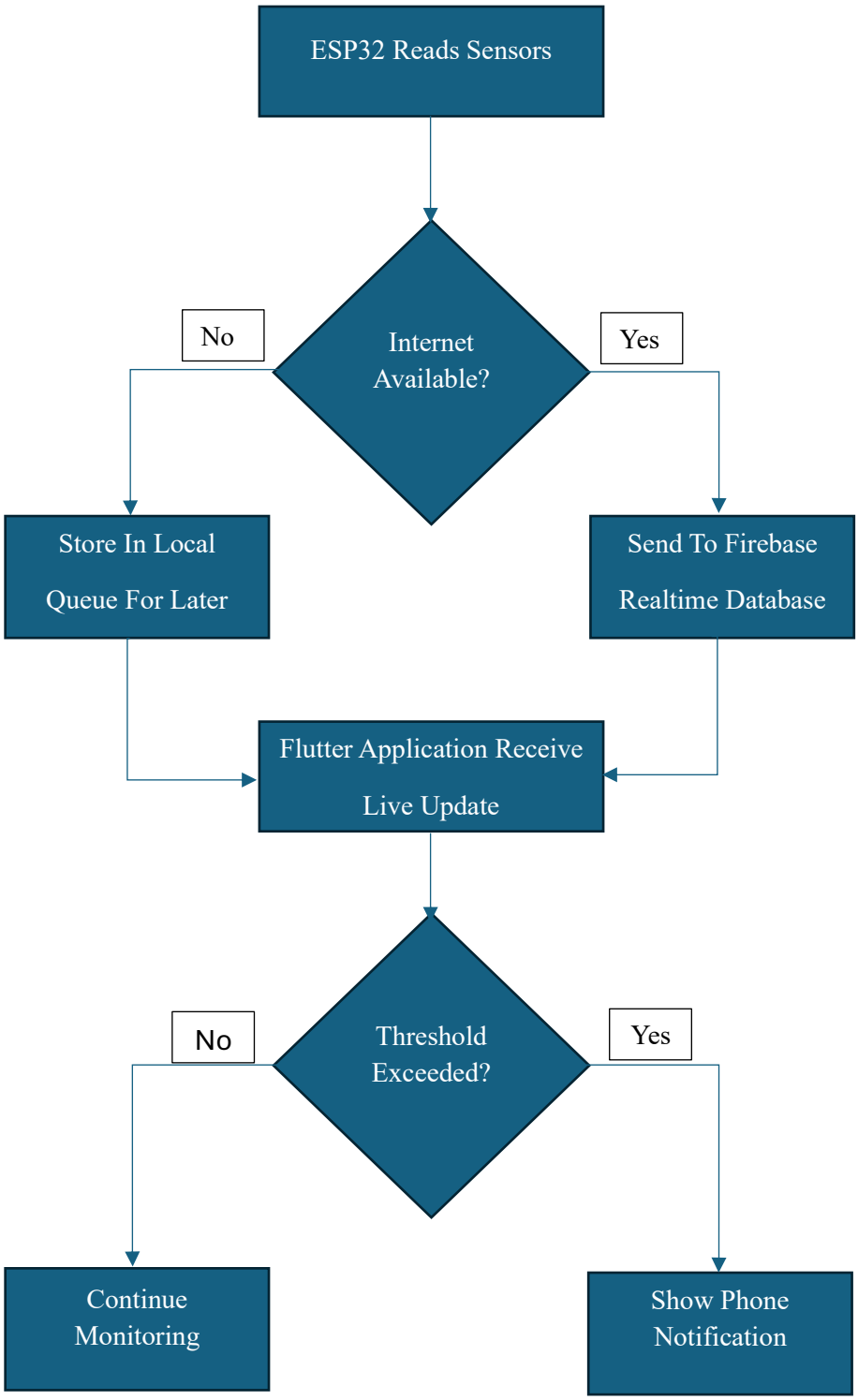


*Figure 8 Firebase Realtime Database & Data Flow from ESP32 to Flutter App*

## 4.4 Flutter Mobile Application

The main user interface is the Flutter mobile app, “Karachi AQMS Pro”. It is written in Dart and use some packages: firebase_database for realtime data, flutter_map for GPS mapping, flowchart for trend graphs, syncfusion_flutter_gauges for radial gauges, flutter_local_notifications for notifications, and flutter_background_service to continue monitoring in background when the app is minimized. It is shown as figure 9 as light and dark mode of AQMS Application.

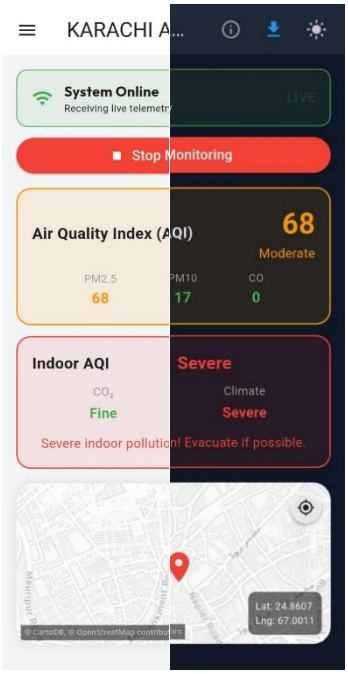


*Figure 9 UI of AQMS Application in light/dark mode*

### 4.4.1 Application Initialization and Notification Channels

Upon starting up our application, we will perform a few functions: initialize firebase, create an Android notification channel named aqms_channel, request notification permission from the user and write the FCM token of that device into /device_tokens in the Firebase database. In addition to all this, we will start a service in the background that will independently receive messages from /sensors/current, check thresholds and generate local notifications even when the application’s main UI has been closed. A persistent notification with an action "Stop Monitoring" will indicate to the user that the background service is running. Notification system in application is being shown in figure 10.

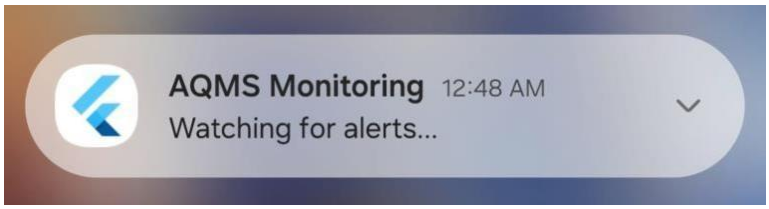


*Figure 10 Notification System*

### 4.4.2 Dashboard UI and Real-Time Data Binding

The AQMS Dashboard widget makes use of 2 Firebase Stream Listeners. One is attached to /sensors/current and sends updates to the live Data map and triggers any UI rebuilds. The second listener tracks the last 300 entries from /sensors/history and fills a list used for trend graphs. The Connectivity Status Banner at the top of the dashboard provides information about whether the internet is connected and whether ESP32 has sent any data. The animated text reading "LIVE" also provides additional assurance of data flowing through the system.


*Figure 11 Live trends of temperature*

There are 8 sensor cards arranged in a grid on the dashboard, with each sensor card colour-coded based on Good, Moderate or Poor statuses based on predetermined threshold limits. By tapping on a sensor card, the user can view more detail about that sensor via a radial gauge as well as a historical trend graph as shown in figure 11.

### 4.4.3 Air Quality Index Calculation Engines

There are two separate indexes of Indoor Air Quality and Outdoor Air Quality implemented based upon the Breeze Technologies model for an Indoor Air Quality Index (IAQI). The real-time local computations of both indices will take place in the Flutter app anytime new sensor data arrives. Thus, at no time, do you need an outside server to obtain a real health-related summary. We can see both readings in the application. Outdoors, the Outdoor AQI conforms to the U.S. EPA standard [9].

#### 4.4.3.1 Outdoor Air Quality Index (AQI)

The outdoor AQI follows the United States Environmental Protection Agency standard [7]. For each pollutant being measured with sensors (PM2.5, PM10, and CO) a sub-index will be computed according to the equation 3.

$$I_p = \frac{IHi - ILo}{BPHi - BPLo} \times ( C_p - \text{BPLo} ) + \text{Ilo} \qquad (3)$$

Where Cp is the measured concentration, BPLo and BPHi are the breakpoints around Cp, and ILo and IHi are corresponding AQI values at those breakpoints. For example, PM2.5, PM10, and CO (8 Hour Average) breakpoint tables are encapsulated as a list of objects of class AqiBreakpoint instantiated in AqiCalculator. Overall, the value of AQI is equal to the maximum of the three individual pollutant sub-indices.

The same procedure is used for determining the PM10 and CO pollution levels using their respective breakpoint tables. Since the app uses instantaneous sensor readings, instead of 24-hour (PM) and 8-hour (CO) averages as recommended, the AQI provided will be an estimate; however, it is still a good live estimate of pollution levels.

#### 4.4.3.2 Indoor Air Quality Index (IAQI)

The IAQI is an evaluation of indoor air on a scale of one (excellent) to six (severe). The overall IAQI score will be the maximum of the two scores that were used to create it. The two component scores are $CO_2$ and the Climate score calculated based on temperature and humidity.

The $CO_2$ score is determined using the simple table from Table 2 based on the $CO_2$ concentration in ppm. The threshold levels were established using the IAQ framework developed by Breeze Technologies.

The Climate score is computed from a temperature–humidity comfort matrix (14 × 11) implemented as a nested map. The matrix assigns a score from 1 to 6 for each combination of temperature (15 – 28 °C) and relative humidity (0 – 100 %, in steps of 10 %). The values were taken directly from the Breeze Technologies IAQI documentation. Listing 4.9 shows a small excerpt of the matrix and the lookup function.

The overall IAQI is then determined by Equation 4.

$$\text{overall\_score} = \max(\text{co2\_score}, \text{climate\_score}) \quad (4)$$

The dashboard displays the overall score and its corresponding label (“Excellent” through “Severe”), along with the two sub-scores, inside a colour coded card. This dual-index presentation makes the system equally relevant for indoor spaces such as classrooms and offices, and outdoor mobile monitoring. The calculation of IAQI for both $CO_2$ and climate is shown by table 2.

*Table 1 Evaluation of $CO_2$, temperature and humidity for IAQI*

| Score | Label | $CO_2$ (ppm) | Temperature (°C) | Relative Humidity (%) |
|---|---|---|---|---|
| 1 | Excellent | ≤ 400 | 20 – 22 | 40 – 50 |
| 2 | Fine | 401 – 1000 | 19 – 23 | 30 – 60 |
| 3 | Moderate | 1001 – 1500 | 18 – 24 | 20 – 70 |
| 4 | Poor | 1501 – 2000 | 16 – 26 | 10 – 80 |
| 5 | Very Poor | 2001 – 5000 | 15 – 28 | 0 – 100 |
| 6 | Severe | > 5000 | < 15 or > 28 | < 10 or > 90 |

### 4.4.4 Interactive Map with Live GPS Tracking

A live map as shown in figure 12, built with flutter map and CartoDB light tiles, displays the device's current position. The marker is updated from the latitude and longitude fields in the Firebase current node. The map auto-centers on new coordinates, and a "My Location" button allows manual re-centering. This feature lays the foundation for pollution heatmapping, as described by Abdullah et al. [13].

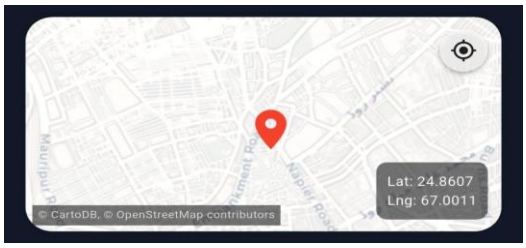


*Figure 12 A GPS Tracking location*

### 4.4.5 Local Notifications, Background Service, and Connectivity Alerts

There are two different types of local notifications: threshold alerts and connectivity alerts. A threshold alert is generated whenever any sensor's value becomes greater than the predefined threshold value as determined by checkCriticalReadings() for the foreground

dashboard and by checkThresholds() for the background service. A connectivity alert will notify the user when the ESP32 goes offline and then notify the user when the ESP32 comes back online after a period of no connection. Debouncing has been implemented (15 seconds) for both cases in order to prevent false notifications. Even if the application is swiped away, the background service ensures the user will receive their notifications.

### 4.4.6 Sensor Detail Page with Live Trend Graphs

A radial gauge with three sections (green, orange and red) and a 10-minute rolling trend is displayed on the Sensor Detail Page. This trend is created using flowchart and provides touch support; if you tap a data point on the trend, you will see a tooltip that contains the corresponding value and timestamp. The y-axis is dynamically scaled to ensure the labels are easily readable by using intelligent interval calculations. When the device is offline, there is an indication in a warning banner that the displayed data could be out of date. The gauge will display the last known value. The live trend is shown in figure 15 and reading user interface is shown in figure 13.

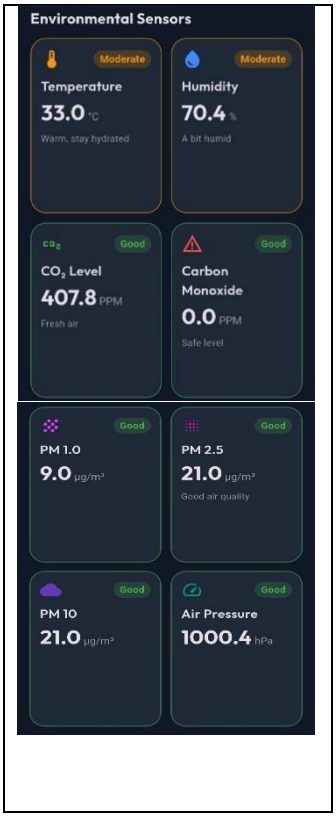


*Figure 13 Reading parameters of sensors in the applications*

### 4.4.7 CSV Export and Data Sharing

For offline analysis and ML dataset preparation, the app exports the last 400 history records to a CSV file. The export To CSV () function serializes history data as rows and writes them to a temporary file and then calls the platform share sheet. This was important for gathering the first training set for the predictive model.

## 4.5 Test Results and live data validation

In deploying a field ready environmental monitoring system, real time sensor diagnostic capabilities are just as critical as data accuracy. Environmental nodes are frequently subjected to variable thermal states, unstable supply voltages, and mechanical vibrations that can compromise sensor connection integrity. To prevent the integration of corrupt metrics into the predictive Machine Learning models, the system incorporates an autonomous hardware and software diagnostic layer.

### 4.5.1 Fault detection and isolation

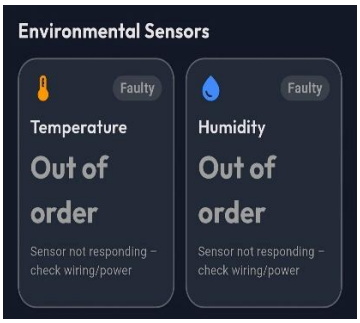


Figure 14 Web Dashboard Alert

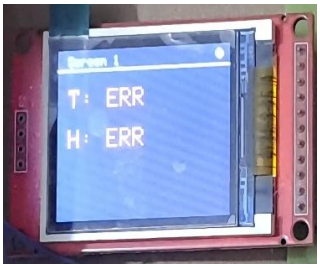


Figure 15 Local TFT Diagnostics

During the experiment, the sensors were removed and replaced by a faulty one to test whether the Fault detection is working or not. Hence, it can be seen in Figure 14 and Figure 15 that both hardware and software detected the malfunction successfully.

### 4.5.2 End to end system performance and data acquisition

When everything is powered up and connected properly, the device automatically loops through a set of different screens on the display. Instead of fitting every single reading onto a tiny screen, the code splits the data into clean, individual pages that are easy to check during field tests without opening a laptop. As it cycles, it gives you a live reading showing gas levels, dust particles, and GPS coordinates while also checking background tasks like

the local IP address and database connection status. This constant rotation basically acts as a quick health check, proving briefly that the hardware is up, running, and collecting data perfectly. This can be seen in Figure 16.


*Figure 16 (a) pressure readings from sensors*


*Figure 16 (b) connected to network and firebase*

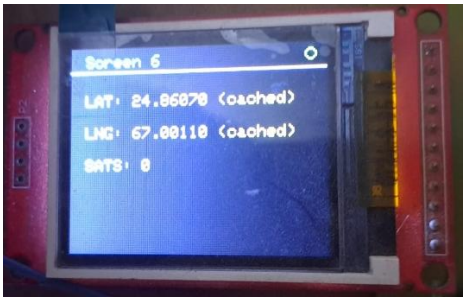


*Figure 16 (c) Location from GPS module*

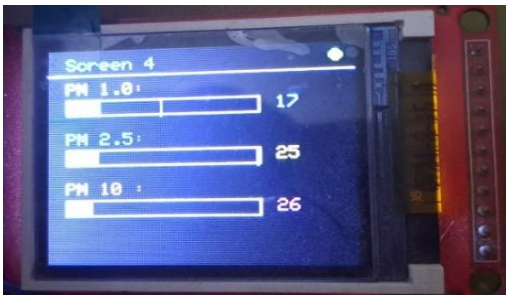


*Figure 16 (d) PM sensor readings*


Figure 16 (e) Carbon dioxide readings


Figure 16 (f) Carbon Monoxide readings

## 4.6 Chapter Summary

This chapter detailed the three-tier software architecture of the air quality monitoring system. The ESP32 firmware implements moving-average filtering, a multi-screen TFT interface, dual-mode connectivity management, an in-memory offline queue for data resilience, threshold-based local and remote alarms, and machine learning. The cloud

backend uses Firebase Realtime Database with a simple scheme that supports real-time synchronization and historical archiving. The Flutter mobile application provides a responsive dashboard, EPA and indoor AQI calculators with full breakpoint and matrix implementations, GPS mapping, local and background notifications, interactive trend graphs, and CSV export. Together, these software components form a complete, resilient pipeline from sensor to user, ready for the validation and machine learning experiments described in Chapter.

# Chapter 5 Machine learning and predictive analysis

## 5.1 Introduction

Modern ambient air quality analysis has rapidly transitioned from sparse, macro-level atmospheric stations to hyper-localized, IoT-enabled indoor and outdoor microclimate monitoring nodes. Human exposure to airborne particulate matter, gaseous pollutants, volatile organic compounds, and adverse thermodynamic indexes occurs primarily within enclosed residential, industrial, and academic facilities. To model these environments effectively, real-time diagnostic hardware must be paired with low-latency, highly predictive statistical and machine learning engines.

The "Universal Air Quality Monitoring and Prediction System" was designed and deployed as an integrated edge-to-cloud hardware-software system to resolve this issue. Operating at high temporal frequencies (1-minute sampling intervals), the physical hardware framework intercepts continuous multi-parameter environmental signatures. However, raw observation is fundamentally reactive. True preventative environmental health frameworks require proactive insights, allowing a system to predict localized environmental deterioration well before critical hazard levels are reached.

This report details the machine learning layer that serves as the predictive intelligence engine of the architecture. Driven by data science methodologies, the system utilizes a rolling historical context window to perform continuous, multi-step autoregressive forecasting. We specifically detail the implementation of Ridge Regression (L2-regularized linear path), explore the performance characteristics that make it uniquely suited to highly correlated indoor environments, and perform an extensive benchmark assessment against Random Forest Regressors and Extreme Gradient Boosting (XGBoost) architectures to determine the optimal trade-off between deployment computational complexity and mathematical predictive power.

## 5.2 End to end system architecture

Our system uses a clean, three-tier architecture to move data from the physical sensors up to our Python analytical engine. Our air quality monitoring system relies on an ESP32 microcontroller for its data acquisition layer, utilizing its dual core processing and built in WiFi to manage localized operations. The node interfaces directly with various sensors via I2C and analog pins to track PM2.5, gas levels, temperature, and humidity. To mitigate random sensor noise, the firmware oversamples the data throughout each minute, computes a clean average, and packages a structured JSON payload every 60 seconds. For ingestion and transport, the ESP32 streams this data over secure HTTPS POST requests to a Firebase Realtime Database, which simultaneously drives live dashboard updates and allows a backend script to archive the JSON entries into CSV logs. Finally, these logs feed into a Python based predictive analytics engine, which handles data cleaning, feature engineering Including time-lagged features and runs the machine learning models to output future air quality predictions. The figure 14 shows how this system works.

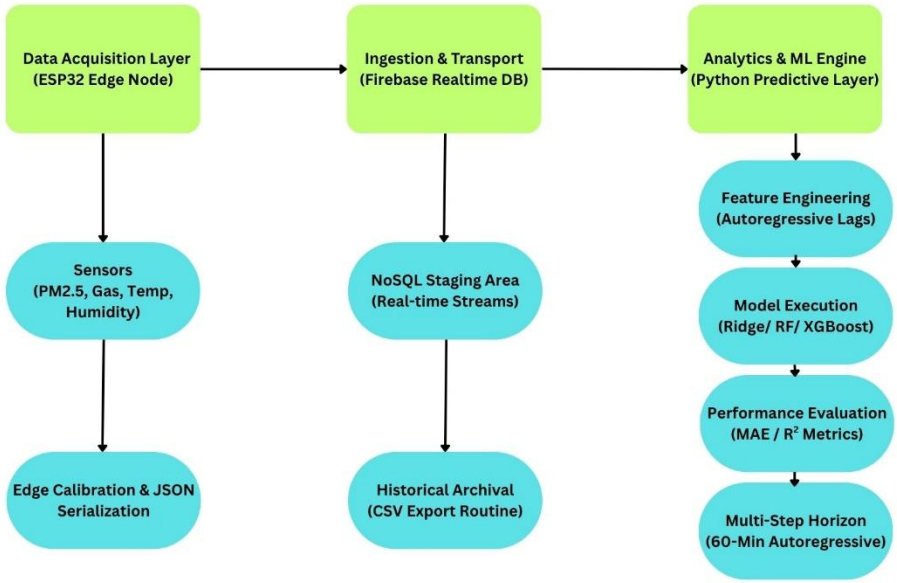


*Figure 14 System Architecture*

## 5.3 Preprocessing and Auto regressive feature engineering

Sensor logs are messy and require cleaning before they can go near a machine learning model.

### 5.3.1 Cleaning the Telemetry

We start by stripping out columns that inject noise or add zero predictive value. This means dropping static spatial invariants like Latitude and Longitude, as well as uncalibrated metrics like incomplete CO (ppm) channels. This streamlines the data so the model focuses purely on the variables that matter.

### 5.3.2 Setting Up the Timeline

We transform the timestamp column into a proper 64-bit datetime index and sort all entries chronologically. We verified our sampling frequency by finding the mathematical mode of the time steps between successive rows as shown in equation 5.

$$\triangleq t = mode(T_t - T_{t-1}) \qquad (5)$$

For our setup, this value is exactly 1 minute, giving us a highly uniform time series to analyze.

### 5.3.3 Preparing Past Data for the Model

Air quality parameters depend heavily on momentum, meaning what happened a few minutes ago tells us a lot about what will happen next. We frame this as an autoregressive forecasting problem. If the current air quality metrics are stored in vector $Y_t$, we pull data from a 3-minute lookback window (k = 3) to capture recent trends as shown in equation 6.

$$X_t = [Y_{t-1}, Y_{t-2}, Y_{t-3}] \qquad (6)$$

This structures our training dataset into input matrix X and target matrix Y. Any incomplete rows caused by this time-shifting step are removed directly via listwise deletion before training.

## 5.4 Theoretical Background of the Models

We analysed three distinct algorithms to see how well they handle our high-frequency, highly correlated datasets.

### 5.4.1 Ridge regression

Standard linear regression uses Ordinary Least Squares (OLS) to minimize error, but it struggles with indoor air parameters. Indoor metrics like temperature, humidity, and gas resistance are heavily collinear, they tend to shift together. This makes standard regression weights unstable, causing the model to overfit to minor data fluctuations.

### 5.4.2 Random forest and XGBoost

Random Forest uses a non-parametric approach, assembling a large group of independent decision trees trained through bootstrap aggregation (bagging).

To keep the trees from duplicating each other and repeating errors, the algorithm checks only a random subset of features at each branch split. The final prediction averages out the individual tree results.

This allows Random Forest to map complex, non-linear environmental thresholds without needing any manual data scaling, though it requires a significant amount of memory to store all the trees.

XGBoost builds decision trees sequentially instead of in parallel. Each new tree focuses entirely on minimizing the residual errors left behind by the trees before it.

## 5.5 Operational Evaluation and Model Selection Trade-offs

When choosing the right predictive model for a live air quality framework, you have to balance pure statistical accuracy against real-world computing constraints. Every algorithm handles data structure, speed, and memory differently, which directly impacts how it runs on your hardware.

Looking first at how these models process data, Ridge Regression relies on a straightforward linear approach. It uses an analytical L2 regularized formula that makes it incredibly stable when dealing with highly correlated indoor metrics like temperature and humidity. Because the math behind it is just a direct matrix inversion, it trains in a fraction of a second and runs instantly. The operational footprint is practically zero since the system only needs to save a single weight vector. The main drawback is that it cannot map non-linear data patterns on its own. If your target metrics experience sudden shocks, Ridge will struggle unless you manually engineer complex interaction features beforehand.

Random Forest shifts away from strict linear paths by using a non-parametric approach. It assembles a massive group of independent decision trees built in parallel through bootstrap aggregation. This structure makes it highly robust against collinear variables because it only samples a random subset of features at each branch split. It is also incredibly capable of capturing sharp, non-linear environmental thresholds without needing manual data adjustments. However, this flexibility comes with a heavy computational cost. Training is quite slow due to the massive tree aggregation process, and checking predictions takes longer because the data has to walk through every single tree path. On top of that, its memory usage is huge because the system has to store the entire forest structure in memory.

XGBoost combines the best of both worlds, making it the ideal selection for our system backend. It uses sequential gradient boosting, where each new decision tree is custom-tailored to fix the errors left by the previous ones. It handles collinear variables easily by applying strict complexity penalties directly inside its optimization function. When it comes to performance, it captures complex non-linear data spikes exceptionally well. Unlike Random Forest, it runs quickly and keeps its memory footprint light because it builds highly compressed, optimized tree structures. This allows it to deliver the high-fidelity accuracy of an ensemble model while maintaining the low-latency response times needed for real-time alerts.

## 5.6 Performance & Benchmarking Results

We split our data chronologically, using the first 80% for training and the final 20% for testing. We avoided random shuffling because it tears apart the time-series continuity, leaking future data into past predictions and skewing test results.

### 5.6.1 Performance Metrics

We monitored model performance using Mean Absolute Error (MAE) and the Coefficient of Determination ($R^2$) in equations 7 and 8 respectively.

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \bar{y}i| \qquad (7)$$

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(yi - \bar{y}i)^2}{\sum_{i=1}^{n}(yi - \bar{y})^2} \qquad (8)$$

*Table 3: Efficiency of ridge regressor*

| Target parameter | $R^2$ | MAE |
|---|---|---|
| Temperature | 0.8970 | 0.2176 |
| Humidity | 0.8090 | 0.8235 |
| $CO_{ppm}$ | 0.8777 | 0.0351 |
| $CO_2ppm$ | 0.8140 | 1.2702 |
| $PM_{2.5}$ | 0.8713 | 0.7002 |
| $PM_1$ | 0.8663 | 1.1352 |
| $PM_{10}$ | 0.8711 | 1.5376 |
| Pressure | 0.8847 | 0.2117 |

*Table 4: Efficiency of random forest*

| Target parameter | $R^2$ | MAE |
|---|---|---|
| Temperature | 0.9965 | 0.2356 |
| Humidity | 0.9892 | 0.8833 |
| $CO_{ppm}$ | 0.9766 | 0.6428 |
| $CO_2 ppm$ | 0.9847 | 0.0343 |
| $PM_{2.5}$ | 0.9722 | 1.1342 |
| $PM_1$ | 0.9681 | 1.4932 |
| $PM_{10}$ | 0.9723 | 1.3014 |
| Pressure | 0.9807 | 0.2343 |

*Table 5: Efficiency of Xgboost*

| Target parameter | $R^2$ | MAE |
|---|---|---|
| Temperature | 0.9966 | 0.2176 |
| Humidity | 0.9906 | 0.8235 |
| $CO_{ppm}$ | 0.9770 | 0.0351 |
| $CO_2 ppm$ | 0.9819 | 1.2702 |
| $PM_{2.5}$ | 0.9715 | 0.7002 |
| $PM_1$ | 0.9669 | 1.1352 |
| $PM_{10}$ | 0.9500 | 1.5376 |
| Pressure | 0.9850 | 0.2117 |

Our benchmarks reveal a clear divide in how these models process environmental data. Ridge Regression performs remarkably well on temperature and humidity. This happens because indoor thermal conditions have a lot of physical momentum. They change slowly and predictably, making them easy for a regularized linear model to track.

However, Ridge struggles heavily with particulate matter and gas resistance. Particulate matter and gases spike sharply based on sudden events, like someone cooking, walking by,

or opening a window. Because Ridge Regression is stuck using a linear path, it smooths out these rapid changes and misses the peaks entirely.

The non-parametric models handle these volatile spikes much better. XGBoost came out on top as the best overall choice for our system backend. Its sequential boosting structure allows it to adjust dynamically to sudden, sharp environmental jumps instead of flattening them out like a linear model or averaging them too broadly like a Random Forest.

## 5.7 Recursive multi step Execution

To forecast 60 minutes ahead using a model trained on short-term 1-minute lags, we use a recursive loop.

1. **First Step:** The model reads actual historical observations from the database to predict the very next minute:
2. **Subsequent Steps:** Since we do not have future sensor readings, the system drops the oldest historical lag, shifts the remaining data down, and feeds its own previous prediction back into the model as an input feature:

We run this loop 60 times to get our full hour-ahead prediction window. While this lets us forecast the future without needing external data streams, it can cause errors to compound. If the model makes a small error at step 2, that error serves as an input feature for step 3, multiplying across the rest of the hour. Regular model retraining and continuous data syncs help keep this tracking error under control.

## 5.8 Chapter Summary

Testing these machine learning approaches shows that high-frequency environmental tracking requires a split strategy to maximize efficiency. Regularized linear systems like Ridge Regression offer low training latency and rock-solid matrix stability, but they miss sudden, non-linear air quality spikes.

For production deployment, we recommend a hybrid architecture:

1. **At the Edge:** Run lightweight Ridge Regression models directly on the ESP32 node to handle short-term, low-latency trend checks without overloading the processor.
2. **In the Cloud:** Route the incoming data streams up to a central server to run our sequential XGBoost model. This handles the complex non-linear tracking and provides accurate, 1-hour look-ahead hazard warnings.

# Chapter 6 Conclusion

This project successfully designed, implemented, and validated a mobile, heterogeneous Air Quality Monitoring, Analysis, and Prediction System (AQMS). By leveraging an ESP32 microcontroller, the system successfully unified data streams from a diverse sensor matrix including the PMS5003 for particulate matter PM2.5, PM10 and PM1.0, the MQ7 for carbon monoxide (CO), the MQ135 for carbon dioxide $CO_2$, the DHT22 for temperature and humidity tracking and BMP280 for pressure. Real-time geospatial coordination was maintained via a NEO-M8N GPS module, while an integrated SIM800L GSM cellular link ensured reliable off-grid telemetry transmission to the cloud database. Locally, the ST7735 TFT display successfully executed rapid color-coded hazard alerts based on dynamic threshold breaches. The analytical and predictive models developed during this research demonstrate that low-cost, high-mobility sensor nodes can complement or substitute for expensive stationary reference stations, providing high-density environmental monitoring essential for urban areas like Karachi.

In the beginning, the background knowledge regarding the rising concerns that elevated the global health risks laid the foundation of the idea leading to exploration of the current literature, finding out that the existing air quality monitors were bulky, did not give any spatial data and were limited to the areas where Wi-Fi was available gave rise to the idea of building a device that provides a reliable solution and bridges the gap.

To fortify durability and protection, the circuit is made on custom PCB which is packed inside compact hardware so that the longevity of the sensitive parts of hardware is maintained even in the harsh outdoor environment. The addition of backup power provision reinforces the reliability and portability of the device.

## 6.1 Achievements

The execution of this project yielded several significant engineering and technical milestones:rdogSuccessfully interfaced and synchronized multiple diverse digital, analog, and serial communication protocols (SPI, UART, Analog) simultaneously onto a single ESP32 microcontroller without processing bottlenecks.

### 6.1.1 Real-Time Geospatial Telemetry Data Pipeline:

Established a robust remote tracking system that accurately binds live air quality data to exact GPS coordinates and continuously updates a cloud database over a cellular GSM network.

### 6.1.2 Custom Printed Circuit Board (PCB) Architecture

Designed, routed, and fabricated a custom PCB layout that successfully mitigated electrical cross-talk, stabilized high-current power spikes from the GSM module, and shrunk the system footprint.

### 6.1.3 Fully Integrated Mobile and Physical Deployment

Developed a user-friendly mobile application for remote monitoring while successfully designing a custom 3D-printed physical chassis to protect the electronics during active on-the-go field testing.

## 6.2 Future Recommendations

### 6.2.1 Integration of Local SD Card Data Logging

To prevent data loss during cellular network outages, a Micro-SD card module should be integrated into the hardware design using the ESP32's secondary SPI pins. This will enable the system to perform offline data backup by writing timestamped sensor logs directly to a CSV file whenever the SIM800L module loses connection, syncing the stored data automatically back to the cloud once network connectivity is restored.

### 6.2.2 Solar Photovoltaic Power Management

For continuous, standalone outdoor operation, the power system should be upgraded by integrating a small 5V/6V solar panel paired with a lithium-ion battery charge controller (such as the TP4056 or CN3791). This configuration will allow the system to recharge its battery pack during daylight hours, eliminating the need for manual recharging or wall adapters and achieving true off-grid mobility.

### 6.2.3 Expansion with High-Precision Electrochemical and NDIR Sensors

To upgrade the system from basic environmental scanning to regulatory-grade tracking, the MQ-series sensors should be replaced or augmented with high-precision electrochemical and Non-Dispersive Infrared (NDIR) sensors. Integrating dedicated, highly selective sensors for gases like Nitrogen Dioxide ($NO_2$), Sulfur Dioxide ($SO_2$), and Carbon Dioxide ($CO_2$) (such as the MH-Z19B NDIR sensor) would eliminate cross-interference and allow the node to calculate a highly accurate, standardized national Air Quality Index (AQI).

# References

[1] M. S. Munir, "Wireless Sensor Network Dependable Monitoring for Urban Air Quality," IEEE Access, vol. 10, pp. 40051-40062, 2022.

[2] S. Kumawat, "IoT Based Smart Environmental Monitoring System Using ESP32 & ThingSpeak," in 2021 International Conference on Innovative Computing (ICIC), Lahore, Pakistan, 2021, pp. 1-6.

[3] World Health Organization, "Ambient (outdoor) air pollution," WHO Fact Sheets, Sep. 2021. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/ambient-(outdoor)-air-quality-and-health

[4] C. A. Pope III and D. W. Dockery, "Health Effects of Fine Particulate Air Pollution: Lines that Connect," Journal of the Air & Waste Management Association, vol. 56, no. 6, pp. 709-742, 2006.

[5] S. Ameer et al., "Urban Air Pollution Monitoring System with Forecasting Models," IEEE Sensors Journal, vol. 19, no. 18, pp. 8104-8114, Sep. 2019.

[6] U.S. Environmental Protection Agency, "Ambient Air Monitoring Technology Information," EPA, 2020. [Online]. Available: https://www.epa.gov/amtic.

[7] World Health Organization, "WHO global air quality guidelines: particulate matter (PM2.5 and PM10), ozone, nitrogen dioxide, sulfur dioxide and carbon monoxide," WHO, Geneva, 2021.

[8] N. Castell et al., "Can commercial low-cost sensor platforms contribute to air quality monitoring and exposure estimates?," Environment International, vol. 99, pp. 293–302, 2017.

[9] S. Hankey and J. D. Marshall, "Land use regression models of on-road particulate air pollution (particle number, black carbon, PM2.5, particle size) using mobile monitoring," Environmental Science & Technology, vol. 49, no. 15, pp. 9194–9202, 2015.

[10] D. J. Sailor, "A review of methods for estimating the urban heat island intensity," International Journal of Climatology, vol. 31, no. 2, pp. 189–199, 2011.

[11] J. Gubbi, R. Buyya, S. Marusic, and M. Palanisami, "Internet of Things (IoT): A vision, architectural elements, and future directions," Future Generation Computer Systems, vol. 29, no. 7, pp. 1645–1660, 2013.

[12] S. Messan, A. Shahud, A. Anis, R. Kalam, S. Ali, and M. I. Aslam, "Air-MIT: Air Quality Monitoring Using Internet of Things," Engineering Proceedings, vol. 20, no. 1, p. 45, 2022.

[13] M. Abdullah, G. Noor, S. Ali, I. Ahmed, and M. I. Aslam, "Development of Cellular IoT-Based, Portable Outdoor Air Quality Monitoring System for Pollution Mapping," Engineering Proceedings, vol. 12, 2025.

[14] Tanzila, S. Ali, M. I. Aslam, I. Ahmed, and A. Ahmed, "Prototyping LoRaWAN-Based Mobile Air Quality Monitoring System for Public Health and Safety," Engineering Proceedings, vol. 118, no. 20, 2025.

[15] A. Spinelle, M. Gerboles, M. G. Villani, M. Aleixandre, and F. Bonavitacola, "Field calibration of a cluster of low-cost available sensors for air quality monitoring. Part A: Ozone and nitrogen dioxide," Sensors and Actuators B: Chemical, vol. 215, pp. 249–257, 2015.

[16] S. Ali, T. Glass, B. Parr, J. Potgieter, and F. Alam, "Low-cost sensor with IoT LoRaWAN connectivity and machine learning-based calibration for air pollution monitoring," IEEE Transactions on Instrumentation and Measurement, vol. 70, pp. 1–11, 2021.

[17] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, no. 7553, pp. 436–444, 2015.

[18] X. Li, L. Peng, Y. Yao, S. Cui, Y. Hu, and C. You, "Long short-term memory neural network for air pollutant concentration predictions: Method development and evaluation," Environmental Pollution, vol. 231, pp. 997–1004, 2017.

[19] H. Mariam, G. Fiza, S. Ali, I. Ahmed, M. I. Aslam, and M. Z. Shakir, "Exploiting Internet of Things to address climate change: Case study and analysis on the perception of

stakeholders in Pakistan," Journal of Climate and Community Development, vol. 3, no. 2, pp. 264–285, 2024.

[20] Espressif Systems, “ESP32 Series Datasheet,” Version 4.7, Dec. 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[21] Plantower, “PMS5003 Series Data Manual,” 2016. [Online]. Available: http://www.plantower.com/en/products/32.html

[22] Guangzhou Aosong Electronics, “DHT22 Humidity & Temperature Sensor Datasheet,” 2015. [Online]. Available: https://www.sparkfun.com/datasheets/Sensors/Temperature/DHT22.pdf

[23] Henan Hanwei Electronics, “MQ-7 Gas Sensor Datasheet.” [Online]. Available: https://www.pololu.com/file/0J311/MQ7.pdf

[24] u-blox, “NEO-M8N Series Datasheet,” 2013. [Online]. Available: https://content.u-blox.com/sites/default/files/products/documents/NEO-6_DataSheet_(GPS.G6-HW-09005).pdf

[25] SIMCom, “SIM800L (MT6261) Hardware Design,” Version 1.01, 2016. [Online]. Available: https://simcom.ee/documents/SIM800L/SIM800L%28MT6261%29_Hardware%20Design_V1.01.pdf

[26] Texas Instruments, “Choosing the Right Buck Converter for Your Application,” Application Report SLVA477A, Oct. 2011. [Online]. Available: https://www.ti.com/lit/an/slva477a/slva477a.pdf